**PROYECTO - Fase I**

**Simulador de Kernel y Gestor de Procesos en Consola (*Shell & Scheduling Engine*)**

## 1. Escenario Hipotético e Introducción

El grupo de Telemática de la Escuela de Ingeniería Informática ha sido contratado por una *startup* de robótica industrial para diseñar el núcleo ejecutor de un nuevo sistema operativo de bajo consumo. La arquitectura del sistema requiere la coexistencia de un entorno interactivo de comandos (CLI) que responda de forma inmediata a los operadores y un motor eficiente de planificación (*Scheduler*) que gestione el tiempo de procesador asignado a los procesos en ejecución.

Para esta primera etapa (Fase 1), se ha encomendado a los equipos de desarrollo el diseño e implementación del **motor de lógica de procesos** y la **Shell interactiva de gestión**. El sistema debe ser capaz de simular la creación de procesos, su ciclo de vida, la asignación de prioridades, el mantenimiento del *Process Control Block* (PCB) y la planificación de la CPU bajo algoritmos apropiativos y no apropiativos, garantizando la concurrencia a través de hilos *POSIX* (*pthread*) y mecanismos de sincronización.

## 2. Objetivos del Proyecto

- **Aplicar conceptos de Concurrencia:** Diseñar una arquitectura multihilo limpia en C/C++ utilizando pthread y exclusión mutua (*pthread_mutex*) para separar la entrada/salida de la consola del motor de ejecución.
- **Modelar el Control de Procesos (PCB):** Estructurar y manipular adecuadamente la información del ciclo de vida de los procesos y sus estados (NUEVO, LISTO, EJECUCIÓN, PAUSADO/BLOQUEADO, FINALIZADO).
- **Implementar Algoritmos de Planificación de CPU:** Programar y contrastar las políticas *Round Robin* (RR) y *Shortest Remaining Time First* (SRTF) / Prioridades Apropiativo, calculando métricas reales de rendimiento.
- **Procesamiento por Lotes (Batch Processing):** Desarrollar un *parser* para la carga masiva y automatizada de archivos de trazas (**workload**).
- **Gestión Profesional de Proyectos de Software:** Utilizar metodologías ágiles mediante *Git*, *GitHub*, etiquetado de *Issues* y tableros tipo *Kanban*.

## 3. Explicación de los componentes de la simulación

La aplicación debe estructurarse obligatoriamente mediante **al menos dos hilos principales de ejecución (*POSIX Threads - pthreads*)** que corren de manera concurrente y se comunican a través de estructuras de datos compartidas (Colas de Procesos) protegidas por cerrojos de exclusión mutua (*pthread_mutex_t*):

| Componente / Hilo | Responsabilidad y Funcionamiento |
|---|---|
| **Hilo CLI (Interfaz de Consola)** | Encargado de la lectura interactiva de comandos del usuario, *parsing* estricto de argumentos, validación de sintaxis, despliegue de tablas de estado, consulta de PCB y visualización del diagrama de *Gantt*. No debe bloquearse ni colapsar por entradas inválidas mientras el procesador simulado ejecuta tareas. |
| **Hilo Motor de Scheduling (CPU)** | Encargado de simular el reloj de la CPU (**clock ticks**), extraer procesos de la cola de LISTOS según la política configurada, simular el consumo de ráfaga (**burst time**), aplicar desapropiación si corresponde, actualizar métricas y registrar el historial de ejecución. |

**4. Lista de Comandos de la CLI y Validación de Entradas**

La Shell interactiva debe *parsear* y validar la sintaxis y los tipos de datos de cada comando. En caso de detectar errores de sintaxis, parámetros faltantes o tipos incorrectos, el Hilo CLI **no debe colapsar**, sino mostrar un mensaje descriptivo de error (por ejemplo: [ERROR]: Cantidad o tipo de parámetros inválido. Uso: create <nombre> <burst> [prioridad]) y solicitar la siguiente entrada.

| Sintaxis del Comando | Parámetros y Validaciones Exigidas | Descripción de Funcionalidad |
|---|---|---|
| ***create*** <nombre> <burst> [prioridad] | <nombre>: *String* sin espacios.<br><burst>: Entero > 0.<br>[prioridad]: Entero entre 1 y 5 (Opcional, Defecto: 3).<br>*Validar cantidad de argumentos y rangos numéricos.* | Crea un nuevo proceso con estado NUEVO, asigna un PID único autoincremental, construye su PCB y lo ingresa a la cola de LISTOS. |
| ***load*** <archivo> | <archivo>: *String* con la ruta del archivo.<br>*Validar existencia y formato del archivo antes de procesar.* | **Modo por lotes:** Lee un archivo de traza e inserta masivamente los procesos en el sistema respetando sus tiempos simulados de llegada (*Arrival Time*). |
| ***ps*** | *Ninguno. Validar que no se envíen argumentos adicionales innecesarios.* | Despliega una tabla con el estado actual de todos los procesos (PID, Nombre, Estado, Ráfaga Restante, Prioridad). |
| ***pcb <PID>*** | PID: Entero positivo.<br>*Validar que el PID sea un número entero y exista en el sistema.* | Muestra el detalle extendido del *Process Control Block* (PCB) del proceso seleccionado (Tiempos de Llegada, Ráfaga Inicial, Restante, Espera $WT$, Retorno $TAT$, Respuesta $RT$). |
| ***set_sched*** <algoritmo> [quantum] | algoritmo: Debe ser exactamente RR o SRTF.<br>quantum: Entero > 0 (Obligatorio/Opcional para RR, Defecto: 2).<br>*Validar coincidencia de nombre de algoritmo y valor positivo de quantum.* | Establece la política de planificación activa del motor de CPU. Puede cambiarse en caliente. |
| ***set_speed*** <ms_por_tick> | <ms_por_tick>: Entero positivo en milisegundos.<br>*Validar que sea un valor numérico válido (ej. entre 50 y 5000 ms).* | Ajusta la velocidad del reloj simulado del sistema (duración del *sleep* real por cada ciclo de CPU). |
| ***start*** | *Ninguno.* | Inicia o reanuda la ejecución del Hilo Motor de Scheduling. |
| ***pause*** | *Ninguno.* | Detiene momentáneamente el reloj de la CPU simulada para permitir la inspección estática del sistema. |
| ***gantt*** | *Ninguno.* | Imprime la representación en texto del Diagrama de Gantt acumulado hasta el instante actual. |
| ***stats*** | *Ninguno.* | Muestra las métricas globales del sistema: Tiempo Promedio de Espera ($\bar{WT}$), Retorno ($\bar{TAT}$), |

| Sintaxis del Comando | Parámetros y Validaciones Exigidas | Descripción de Funcionalidad |
|---|---|---|
| | | Respuesta ($\bar{RT}$) y porcentaje de uso de CPU. |
| ***gui*** *(OPCIONAL / BONUS)* | *Ninguno.* | Abre una ventana gráfica auxiliar (utilizando C++ y una librería gráfica como Raylib, SFML o Qt) para visualizar el estado del sistema en tiempo real. |
| ***help*** | *Ninguno.* | Muestra el manual de usuario rápido de la CLI. |
| **exit** | *Ninguno.* | Libera la memoria dinámica, detiene de forma limpia los hilos y cierra la simulación. |

**5. Formato de Salida por Consola Esperado**

Para asegurar una evaluación uniforme, las salidas de los comandos clave deben seguir el siguiente formato:

**Salida del comando ps**

```
PID    NOMBRE          ESTADO       BURST RESTANTE    PRIORIDAD
-------------------------------------------------------------
1      Proceso_A       RUNNING      3s                2
2      Proceso_B       READY        5s                1
3      Proceso_C       READY        2s                3
4      Proceso_D       TERMINATED   0s                2
```

**Salida del comando pcb 1**

```
================================================
        CONTROL BLOCK DE PROCESO (PCB) - PID: 1
================================================
Nombre del Proceso : Proceso_A
Estado Actual      : RUNNING
Prioridad          : 2
Tiempo de Llegada  : 0 ticks
Ráfaga Inicial     : 5 ticks
Ráfaga Restante    : 3 ticks
Tiempo de Espera   : 2 ticks
Tiempo de Retorno  : N/A (En ejecución)
Tiempo de Respuesta: 0 ticks
================================================
```

**Ejemplo de Mensajes de Error por Validación en CLI**

```
> create Proceso_A
[ERROR DE SINTAXIS]: Parámetros insuficientes.
Uso correcto: create <nombre> <burst_time> [prioridad]

> pcb abc
[ERROR DE TIPO]: El PID debe ser un número entero positivo.
```

```
> set_sched FIFO
[ERROR DE PARAMETRO]: Algoritmo no reconocido. Algoritmos válidos: RR,
SRTF.
```

**Salida de los comandos gantt y stats**

```
DIAGRAMA DE GANTT (0 - 8 ticks):
[0-2s: P1] -> [2-4s: P2] -> [4-6s: P3] -> [6-8s: P1] -> [IDLE]

ESTADÍSTICAS DEL SISTEMA:
- Procesos Finalizados      : 2 / 4
- Tiempo Promedio de Espera : 2.50 ticks
- Tiempo Promedio de Retorno: 6.00 ticks
- Uso de CPU                : 100.0%
```

## 6. Consideraciones de la Simulación y Manejo del Tiempo

Para lograr una sensación de simulación en "cámara lenta controlada" sin distorsionar el rigor matemático del cálculo de métricas del SO, los estudiantes deberán atender las siguientes reglas:

- **Reloj Simulado Discreto (****Clock Ticks****):** El tiempo del sistema no se mide con el reloj real de la laptop o computador a usar, sino con una variable entera acumulativa (*clock_ticks*) que incrementa en $+1$ por cada ciclo de CPU consumido.
- **Retardo Real (Speed Control):** Por cada *tick* de CPU, el Hilo Motor ejecutará un retardo real mediante usleep() o std::this_thread::sleep_for() equivalente al parámetro configurado en set_speed (Por defecto: 1000 ms = 1 segundo real).
- **Sincronización mediante Mutex:** Las colas de procesos y las variables globales deben protegerse obligatoriamente con pthread_mutex_lock y pthread_mutex_unlock cada vez que el Hilo CLI o el Hilo Motor lean o modifiquen el estado de los procesos.

## 7. Escenarios de Prueba para Validación

### Escenario 1: Simulación Manual e Interactiva

- Ejecutar la aplicación y configurar: set_sched RR 2.
- Ajustar velocidad: set_speed 1000.
- Crear 3 procesos:
  - create P1 5 2
  - create P2 3 1
  - create P3 2 3
- Ingresar start. Inspeccionar la alternancia de procesos cada 2 segundos enviando el comando ps.
- Probar validación de errores ingresando comandos mal formados (ej. create BadProcess o pcb xyz) y verificar que el sistema no colapse.
- Durante la ejecución, pausar con pause, consultar el PCB de un proceso con pcb 1, cambiar a set_sched SRTF y reanudar con start.
- Verificar al finalizar los comandos gantt y stats.

### Escenario 2: Carga en Lote (Batch Mode) con Traza de Trabajo

Crear un archivo de texto llamado *workload_test.txt* con el siguiente formato estructurado:

```
# NOMBRE, BURST_TIME, PRIORIDAD, ARRIVAL_TIME
Job_A, 4, 2, 0
Job_B, 2, 1, 1
Job_C, 1, 3, 3
```

- Iniciar el simulador y ejecutar: set_sched SRTF.

- Cargar el archivo: load *workload_test.txt*.
- Escribir start y dejar correr la simulación.
- Verificar la desapropiación inmediata en $t = 1$ cuando llega Job_B ($BT = 2$) desplazando a Job_A ($BT = 3$ restante).
- Validar que la secuencia reportada en gantt coincida exactamente con: [0-1s: Job_A] -> [1-3s: Job_B] -> [3-4s: Job_C] -> [4-7s: Job_A].

**8. Entregables y Artefactos a Desarrollar**

- **Código Fuente Completo en C/C++:** Organizado modularmente en archivos cabecera (.h / .hpp) y fuente (.c / .cpp) con su respectivo Makefile o CMakeLists.txt. **Todo el código debe estar debidamente documentado.**
- **Repositorio en GitHub con Gestión de Proyecto:**
  - Uso activo de commits por cada integrante.
  - Creación de *Issues* categorizados para la asignación de módulos.
  - Tablero Kanban de GitHub Projects organizado en columnas (*To Do*, *In Progress*, *Done*).
- **Informe Técnico de Arquitectura (PDF):** Documento breve que describa la estructura modular del código, diagrama de hilos/sincronización, referencias bibliográficas y un **Anexo Obligatorio de Declaración del Uso de Inteligencia Artificial** (detallando herramienta utilizada, prompts principales y cómo gestionaron/validaron los resultados aportados, según lo estipulado en la Sección 9).

**9. Requisitos Técnicos, Condiciones y Restricciones**

- **Lenguaje y Compilación:** Lenguaje C (estándar C11) o C++ (estándar C++17 en adelante). Compilador gcc / g++ con soporte para hilos POSIX (-pthread).
- **Plataforma:** Entornos basados en POSIX (Linux, WSL en Windows, macOS).
- **Conformación de Equipos:** Grupos de máximo 3 personas (o 2 según distribución del curso).
- **Defensa y Evaluaciones (10% de la nota total del curso = 2 puntos sobre 20):**
  - **Evaluación Grupal y Código (50% = 1.0 pt):** Demostración del funcionamiento correcto de los algoritmos, la CLI (incluyendo robustez frente a errores) y la gestión del proyecto en GitHub.
  - **Defensa Individual (50% = 1.0 pt):** Preguntas individuales sobre el código a cada integrante. La presencia de todos los miembros en la defensa es de carácter **estrictamente obligatorio**.
- **Uso Ético de Inteligencia Artificial:** No está prohibido el uso de herramientas de IA (*ChatGPT*, *Claude*, *Copilot, etc.*). Sin embargo, en caso de utilizarlas, deben ser empleadas de forma ética y con proceso de pensamiento crítico, indicando actuación de rol de tutor socrático o asistente de depuración. Los equipos deberán incluir en el informe técnico (Sección 8) un anexo declarando: herramienta utilizada, prompts principales y cómo validaron y gestionaron los resultados aportados por la herramienta.

**10. Cronograma y Criterios de Evaluación**

| Hito / Entregable | Fecha de Entrega | Ponderación (%) |
|---|---|---|
| **Entrega y presentación de Proyecto Fase 1 (Código, GitHub, Informe)** | **Miércoles 11/11/2026 2:00pm (Semana 9)** | **10% Nota Definitiva (2.0 Puntos / 20 pts)** |

**Criterios de Evaluación Detallados (Escala Base de 2.0 Puntos)**

| Criterio de Evaluación | Descripción | Puntaje Máximo |
|---|---|---|
| **Arquitectura Concurrente y Sincronización** | Uso correcto de pthread, separando Hilo CLI de Hilo Motor con protección de exclusión mutua mediante *pthread_mutex* sin condiciones de carrera. | 0.50 pts |
| **Lógica de Procesos, PCB, CLI y Validaciones** | Implementación del ciclo de vida de los procesos, comandos de consulta (ps, pcb, gantt, stats), formateo adecuado y **validación estricta de entradas/parámetros con manejo limpio de errores sin colapsos**. | 0.50 pts |
| **Algoritmos de Planificación y Batch** | Funcionamiento exacto de *Round Robin* y *SRTF*, lectura de archivos por lotes (load) y cálculo de métricas matemáticas. | 0.50 pts |
| **Gestión de Proyecto en GitHub (Kanban)** | Evidencia de trabajo colaborativo, commits individuales distribuidos, gestión de *Issues* y tablero Kanban activo. | 0.30 pts |
| **Informe Técnico y Documentación (inc. Anexo IA)** | Claridad de la arquitectura expresada en el documento, código documentado y anexo de declaración de uso ético de IA. | 0.20 pts |
| **TOTAL BASE** | **Cumplimiento total de los requerimientos obligatorios de la Fase 1.** | **2.00 pts** |
| **BONUS OPCIONAL: Interfaz Gráfica (GUI)** | Despliegue de ventana interactiva mediante Raylib, SFML, Qt o ImGui para visualizar la lista de procesos y diagrama de Gantt. | **1.00 pto** |